



Python Programming - 2301CS404

Lab - 12

Roll No : 514

Name : Meet Indoriya

In []:

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [14]: try:
div = 10//0 # Divison With Zero
print(div)
except ZeroDivisionError:
    print("ZeroDivisionError Occur")

try:
    a = int("abc") # Value not Conveted In Valid DataType
except ValueError:
    print("Value Error Occur")

try:
    b = "10" + 5
except TypeError: # Invalid DataType Operation
    print("TypeError Occur")
```

ZeroDivisionError Occur
Value Error Occur
TypeError Occur

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

```
In [18]: li = [10,20,30,40,50,60]
di = {1:10 , 2:20 , 3:30}
try:
    c = li[6]
except IndexError: # Index Not Found In Iterable Object
    print("Index Error Occur")

try:
    d = di[4] # Key Not Found In Dictionary
except KeyError:
    print("Key Error Occur")
```

Index Error Occur
Key Error Occur

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [26]: try:
fp = open("Readme.md","r")
data = fp.read()
except FileNotFoundError: # If File Not Found
    print("FileNotFoundError")
else:
    fp.close()

try:
    import matlab
except ModuleNotFoundError: # Module Not Found
    print("Not Imported")
```

FileNotFoundError
Not Imported

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [46]: try:
a= 10//0
except Exception as err:
    print(type(err).__name__ + ": " + str(err))
```

ZeroDivisionError: integer division or modulo by zero

05) WAP to demonstrate else and finally block.

```
In [59]: try:
        for i in range(1,11,1):
            print(i)
        except Exception as err:    # While Error Occur The Except Is Run
            print(err)
        else:    # If Try Runs Complete Without Break Statement then go to else Block.
            print("If Block Executed Successfully")
        finally:
            print("finally Execute Always")
```

```
1
2
3
4
5
6
7
8
9
10
If Block Executed Successfully
finally Execute Always
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```
In [58]: def divided(a,b):
        try:
            c = a/b
        except Exception as err:
            # Standard Format ClassName: Error Message
            print(type(err).__name__ + ": " + str(err))
        else:
            print(c)
        finally:
            print("Operation Completed Successfully")
        divided(10,2)
```

```
5.0
Operation Completed Successfully
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```
In [21]: class NegativeNumberError(Exception):
        pass

n = int(input("Enter The Item Number: "))
total = 0
try:
    for i in range(1,n+1,1):
        num = int(input("Enter The Number"))
        if (num >= 0):
            total += num
        else:
            raise NegativeNumberError("Negative Number Error")
except NegativeNumberError as err:
```

```

    print(err)
else:
    print("Total Bill Is: " + str(total))

```

Negative Number Error

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer. x

You should use a try statement to inform the user when the values they entered cannot be converted.

```

In [2]: grade = input("Enter grades: ")

try:
    grades = [int(grade) for grade in grade.split(",")]
    print("Grades list:", grades)

except ValueError:
    print("Error: Please enter only numbers separated by commas.")

```

Grades list: [10, 20, 30, 40]

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```

In [6]: total = 0
try:
    for i in range(1, 6):
        marks = int(input("Enter marks: "))

        # assert is keyword if condition returns False, AssertionError is raised
        assert 0 <= marks <= 100, "Marks should be between 0 and 100."
        total += marks
    print("Valid marks entered")
    average = total / 5
    print("Average Of Marks:", average)

except AssertionError as e:
    print("AssertionError:", e)

```

AssertionError: Marks should be between 0 and 100.

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```
In [15]: class WeakPasswordException(Exception):
        pass

        try:
            password = input("Enter password: ")
            if len(password) < 8 or not any(ch.isdigit() for ch in password): # Check
                raise WeakPasswordException("Password must have at least 8 characters an

            print("Correct")

        except WeakPasswordException as e:
            print("WeakPasswordError:", e)
```

Correct

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```
In [23]: import math

        try:
            num = float(input("Enter a number: "))
            if num < 0:
                raise NegativeNumberError("square root of a negative number") # class C
            print("Square root:", math.sqrt(num))
        except NegativeNumberError as err:
            print(type(err).__name__ + ": " + str(err))
```

NegativeNumberError: square root of a negative number

In []: